

OpenID Connect — Request & Response Flow

Authorization Code flow with PKCE · a hand-rolled Spring Boot demo

Generated 2026-06-10

Auth Server :9000

Client App :8080

Resource Server :9100

RS256 · PKCE S256

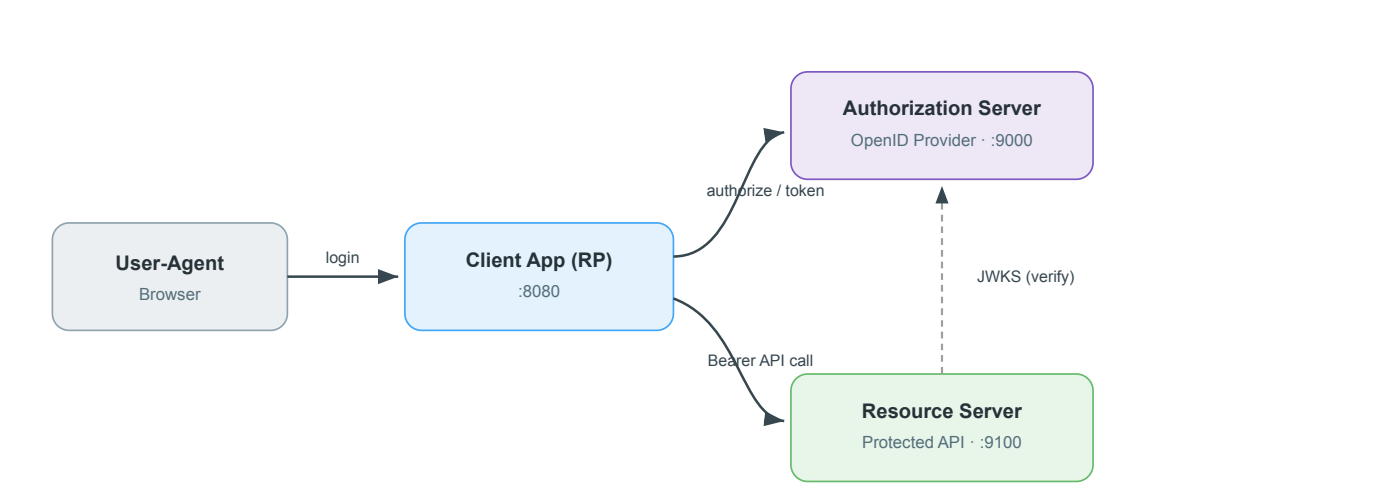
Refresh-token rotation

Every request and response in this document was captured live with `curl` against the three running services. Tokens and codes are real values from one end-to-end run (truncated where long).

1 · Roles

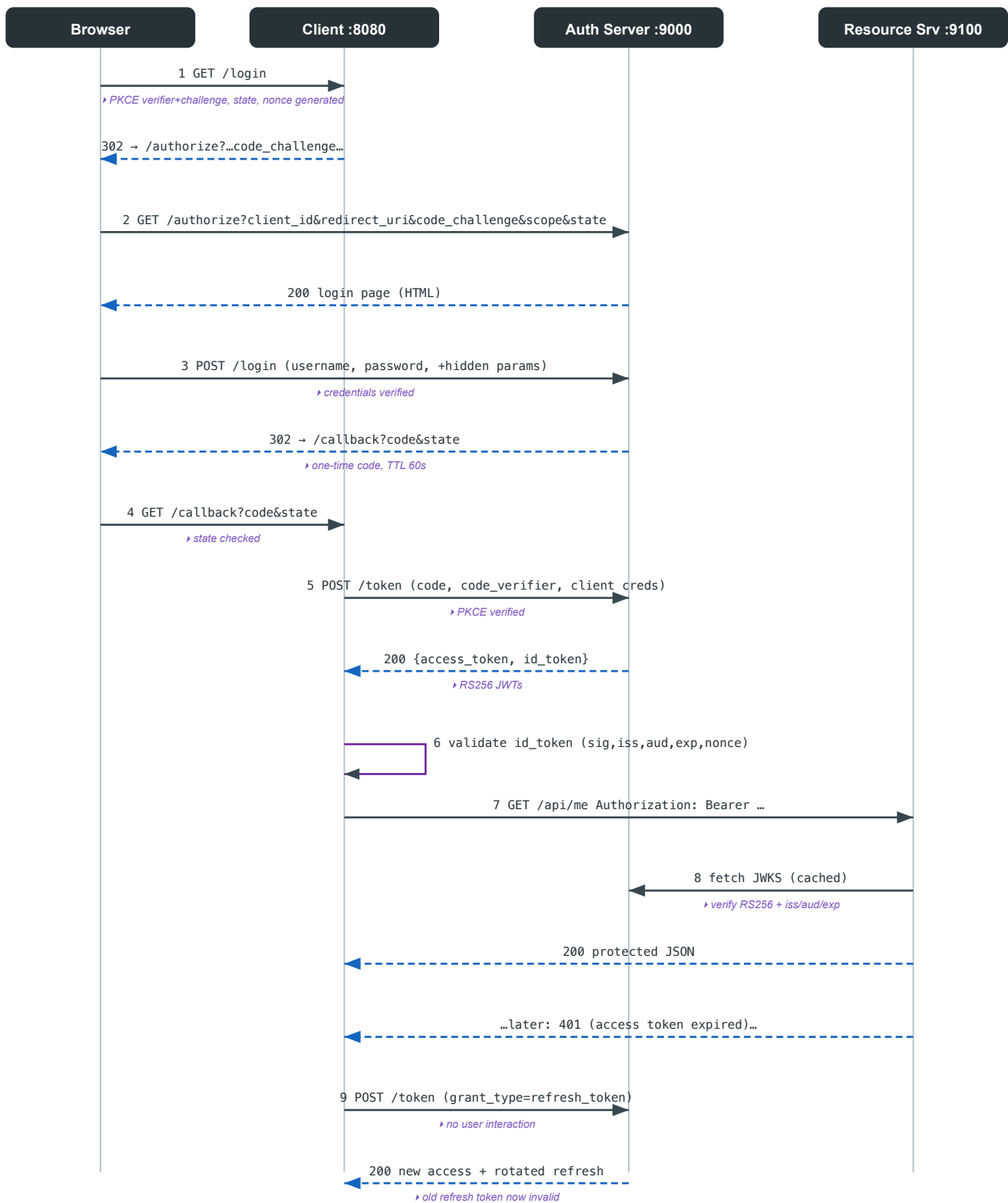
Component	Spec role	Port	Responsibility
authorization-server	OpenID Provider (OP)	9000	Authenticates users; issues authorization codes, ID & access tokens; publishes discovery + JWKS.
client-app	Relying Party (RP)	8080	Drives the browser through login; performs the PKCE token exchange; validates the ID token; calls the API.
resource-server	Resource Server	9100	Validates the access-token JWT (signature, iss, aud, exp) and enforces scope on <code>/api/*</code> .

2 · Architecture



Solid arrows are direct HTTP calls; the dashed arrow is the resource server fetching the provider's public keys (JWKS) to verify tokens offline — it never sees the user's credentials.

3 · Sequence of the full flow



Dashed blue arrows are responses. The PKCE code_verifier stays inside the client until the back-channel token call (step 5), so an intercepted authorization code is useless without it.

4 · Step-by-step with curl

1 Discovery — find the provider's endpoints

The client (or resource server) bootstraps by fetching the OpenID Provider's well-known configuration. Every URL below is derived from the single *issuer* value, so there is one source of truth.

REQUEST

```
curl -s http://localhost:9000/.well-known/openid-configuration
```

RESPONSE

```
{
  "subject_types_supported": [
    "public"
  ],
  "claims_supported": [
    "sub",
    "iss",
    "aud",
    "exp",
    "iat",
    "nonce",
    "auth_time",
    "name",
    "preferred_username",
    "email",
    "email_verified"
  ],
  "authorization_endpoint": "http://localhost:9000/authorize",
  "token_endpoint_auth_methods_supported": [
    "client_secret_basic",
    "client_secret_post"
  ],
  "issuer": "http://localhost:9000",
  "scopes_supported": [
    "openid",
    "profile",
    "email",
    "api.read"
  ],
  "grant_types_supported": [
    "authorization_code",
    "refresh_token"
  ],
  "response_types_supported": [
    "code"
  ],
  "token_endpoint": "http://localhost:9000/token",
  "code_challenge_methods_supported": [
    "S256",
    "plain"
  ],
  "id_token_signing_alg_values_supported": [
    "RS256"
  ],
  "userinfo_endpoint": "http://localhost:9000/userinfo",
  "jwks_uri": "http://localhost:9000/.well-known/jwks.json"
}
```

2 JWKS — fetch the public signing keys

The provider publishes its RSA public keys as a JWK Set. Clients use these to verify ID-token signatures; the resource server uses them to verify access-token signatures. The kid ties a token header to a key.

REQUEST

```
curl -s http://localhost:9000/.well-known/jwks.json
```

RESPONSE

```
{
  "keys": [
    {
      "kty": "RSA",
      "e": "AQAB",
      "kid": "294b8ecc-c6ee-4c42-843d-966d5faebb1f",
      "n": "uws-I01fV0b03kWATy2ANZ1vb1VRedpLIFuuio92Bwc8wcdgqS0-4CF2c6d8GNZY-3hnTJwyVJUo6_fh-01dzpHD9uRkIcnWwUX5wSsWzDDq3e00yph1V18ubVzxza3HFNM-cucAF6nTFFb371l0UjhsWSl0Lw7GcDm0sUc809giBZ9kLzyKXR37VjcLn58VzwpK_5Vm7ZyvUpFD0IQykHEKo0kAf2nsUpY9gRr7fGyiZk4ezPDqsMKWJG63gb34N0bxbSLkRlLmyMs6QISbB7FW5HL5YTa7VjDu6_SRL6R4fMv3rpmi5veVk4YmTWbBK0F5l1Xb69Xv-EeNZiUn6Q"
    }
  ]
}
```

3 Authorization request — redirect the user to sign in

The client generates a PKCE pair and redirects the browser to /authorize. The code_challenge is BASE64URL(SHA256(code_verifier)); the verifier never leaves the client until step 5.

```
code_verifier xkQEVovM1bgmzSqGEek5f0SX43cyKdGnRnwisfZdBFI
code_challenge Bf0cALueanglhTUHsxqhris0snpHium45fFNo1vqe6E
state d7721ddc4e1503a4
nonce 9efb75cd979966c0
```

REQUEST

```
curl -s -G http://localhost:9000/authorize \
  --data-urlencode 'response_type=code' \
  --data-urlencode 'client_id=demo-client' \
  --data-urlencode 'redirect_uri=http://localhost:8080/callback' \
  --data-urlencode 'scope=openid profile email api.read' \
  --data-urlencode 'state=d7721ddc4e1503a4' \
  --data-urlencode 'nonce=9efb75cd979966c0' \
  --data-urlencode 'code_challenge=Bf0cALueanglhTUHsxqhris0snpHium45fFNo1vqe6E' \
  --data-urlencode 'code_challenge_method=S256'
```

RESPONSE

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!-- The provider renders its login page. Every authorization-request
      parameter is carried forward as a hidden field so it survives the
      POST /login submission. -->
<form method="post" action="/login"> ... </form>
```

4 Login — user authenticates, provider issues a code

The user submits credentials. The provider validates them, mints a single-use authorization code (TTL 60s) bound to the client, redirect URI, scope, nonce and PKCE challenge, then 302-redirects back to the client. The state is returned unchanged so the client can detect CSRF.

REQUEST

```
curl -s -i http://localhost:9000/login \  
  -d 'username=alice' -d 'password=password' \  
  -d 'client_id=demo-client' \  
  -d 'redirect_uri=http://localhost:8080/callback' \  
  -d 'scope=openid profile email api.read' \  
  -d 'state=d7721ddc4e1503a4' -d 'nonce=9efb75cd979966c0' \  
  -d 'code_challenge=Bf0cALueanglhTUHsxqhris0snpHium45fFNo1vqe6E' \  
  -d 'code_challenge_method=S256'
```

RESPONSE

```
HTTP/1.1 302  
Location: http://localhost:8080/callback?code=285743145bdb490faa64abeb10429fbd&state=d7721ddc4e1503a4
```

5 Token exchange — swap code (+verifier) for tokens

The client calls the back-channel /token endpoint, authenticating with HTTP Basic (client_secret_basic) and presenting the original code_verifier. The provider re-derives the challenge from the verifier, confirms it matches what was stored in step 4, and only then returns an **access token** + **ID token** (RS256 JWTs) and a **refresh token** (opaque, for later renewal).

REQUEST

```
curl -s -X POST http://localhost:9000/token \  
  -u 'demo-client:demo-secret' \  
  -d 'grant_type=authorization_code' \  
  -d 'code=285743145bdb490faa64abeb10429fbd' \  
  -d 'redirect_uri=http://localhost:8080/callback' \  
  -d 'code_verifier=xkQEVovM1bgmzSqGEek5f0SX43cyKdGnRnwisfZdBFI'
```

RESPONSE

```
{  
  "access_token": "eyJraWQiOiIyOTRiOGVjYy1j\u02026U3bVJgDqZd1Q",  
  "token_type": "Bearer",  
  "expires_in": 300,  
  "id_token": "eyJraWQiOiIyOTRiOGVjYy1j\u02026B0L1a54MmEcg",  
  "refresh_token": "h4CUv3qU3jxv\Fsw_0RMVAzQAwkIXuSuFP8WA0qjaM",  
  "scope": "openid profile email api.read"  
}
```

5a Anatomy of the tokens (decoded)

Both JWTs are signed with RS256. Decoding the Base64URL segments shows why each token exists: the **ID token**'s audience is the client (it proves *who* the user is, and echoes the nonce), while the **access token**'s audience is the resource server (it grants *what* the bearer may do, via scope).

ACCESS TOKEN (HEADER + CLAIMS)

```
# header is identical for both (selects the verification key by kid)
{
  "kid": "294b8ecc-c6ee-4c42-843d-966d5faebb1f",
  "typ": "JWT",
  "alg": "RS256"
}

# ACCESS TOKEN payload – aud = resource-server
{
  "sub": "user-001",
  "aud": "resource-server",
  "scope": "openid profile email api.read",
  "iss": "http://localhost:9000",
  "exp": 1781031272,
  "iat": 1781030972,
  "jti": "2b964d54-46ae-4a46-b471-7e4446aa4f89",
  "client_id": "demo-client"
}
```

ID TOKEN CLAIMS

```
# ID TOKEN payload – aud = demo-client, carries identity + nonce
{
  "sub": "user-001",
  "aud": "demo-client",
  "email_verified": true,
  "auth_time": 1781030972,
  "iss": "http://localhost:9000",
  "name": "Alice Anderson",
  "preferred_username": "alice",
  "exp": 1781031272,
  "iat": 1781030972,
  "nonce": "9efb75cd979966c0",
  "email": "alice@example.com"
}
```

6 UserInfo — fetch claims with the access token

Optionally, the client calls the provider's `/userinfo` endpoint with the access token. The returned claims are filtered by the granted scopes: profile adds name/username, email adds the email claims.

REQUEST

```
curl -s http://localhost:9000/userinfo \
-H "Authorization: Bearer $ACCESS_TOKEN"
```

RESPONSE

```
{
  "sub": "user-001",
  "email_verified": true,
  "name": "Alice Anderson",
  "preferred_username": "alice",
  "email": "alice@example.com"
}
```

7 Public endpoint — no token required

The resource server exposes one unauthenticated route for contrast. The bearer-token filter is wired only to `/api/*`, so the root is reachable with no credentials.

REQUEST

```
curl -s http://localhost:9100/
```

RESPONSE

```
{
  "service": "resource-server",
  "hint": "Call /api/me or /api/messages with a Bearer access token",
  "status": "up"
}
```

8 Protected API — call `/api/me` with the Bearer token

Now the client calls the protected API. The resource server's servlet filter extracts the Bearer token, verifies its RS256 signature against the cached JWKS, and enforces the expected `iss`, `aud` (`resource-server`) and `exp`. The validated claims drive the response.

REQUEST

```
curl -s http://localhost:9100/api/me \
-H "Authorization: Bearer $ACCESS_TOKEN"
```

RESPONSE

```
{
  "clientId": "demo-client",
  "subject": "user-001",
  "scopes": [
    "email",
    "openid",
    "profile",
    "api.read"
  ]
}
```

9 Scope-protected data — /api/messages (needs api.read)

This endpoint additionally enforces *fine-grained authorization*: it requires the `api.read` scope, which our access token carries.

REQUEST

```
curl -s http://localhost:9100/api/messages \
-H "Authorization: Bearer $ACCESS_TOKEN"
```

RESPONSE

```
{
  "owner": "user-001",
  "messages": [
    {
      "text": "Welcome, your token is valid!",
      "from": "system",
      "id": 1
    },
    {
      "text": "This data is protected by an access token.",
      "from": "system",
      "id": 2
    }
  ],
  "servedAt": "2026-06-09T18:49:33.104699Z"
}
```

10 Token refresh — renew without re-login

Access tokens are short-lived (300s). When one expires, the API returns `401` and the client trades its **refresh token** at `/token` for a fresh set — no user interaction. The refresh token is opaque and **stored server-side**, which is what lets the OP *rotate* it: each use is single-use and returns a new refresh token. The renewed access token works immediately at the resource server (verified: `GET /api/me` → **200**).

REQUEST

```
curl -s -X POST http://localhost:9000/token \
-u 'demo-client:demo-secret' \
-d 'grant_type=refresh_token' \
-d 'refresh_token=h4CUv3qU3jxvL_Fs...A0qjaM'
```

RESPONSE

```
{
  "access_token": "eyJraWQiOiIyOTRiOGVjYy1j\u2026ht1wTJqIUZHw",
  "token_type": "Bearer",
  "expires_in": 300,
  "id_token": "eyJraWQiOiIyOTRiOGVjYy1j\u2026B1175gH7yB8Q",
  "refresh_token": "-LZBhQjT15MPDNRqsVZMt8DC_SScI_9uN8hwU-lIC20",
  "scope": "openid profile email api.read"
}
```


10a Rotation — the old refresh token is now dead

Because the OP consumed the presented refresh token and issued a new one, replaying the *old* value fails. This single-use rotation turns a leaked-and-replayed refresh token into a detectable error.

REQUEST

```
# reuse the PREVIOUS refresh token (h4CUv3qU3jxvL_Fs...A0qjaM)
curl -s -X POST http://localhost:9000/token \
-u 'demo-client:demo-secret' \
-d 'grant_type=refresh_token' \
-d 'refresh_token=h4CUv3qU3jxvL_Fs...A0qjaM'
```

RESPONSE

```
{
  "error_description": "Unknown or already-used refresh token",
  "error": "invalid_grant"
}
```

5 · Error responses

The same endpoints, exercised with a missing or under-privileged token. These show how the resource server distinguishes *authentication* (401) from *authorization* (403).

E1 401 Unauthorized — no Bearer token

Calling a protected route without a token. Per RFC 6750 the server replies with a `WWW-Authenticate: Bearer` challenge header.

REQUEST

```
curl -s -i http://localhost:9100/api/me
```

RESPONSE

```
HTTP/1.1 401
WWW-Authenticate: Bearer error="missing_token",
  error_description="A Bearer access token is required"

{
  "error": "missing_token",
  "error_description": "A Bearer access token is required"
}
```

E2 403 Forbidden — token valid but scope insufficient

Here a second user (bob) logged in requesting only `openid profile` — no `api.read`. The token is perfectly valid and `/api/me` would succeed, but `/api/messages` rejects it with `insufficient_scope`.

REQUEST

```
# token obtained with scope='openid profile' (no api.read)
curl -s -i http://localhost:9100/api/messages \
-H "Authorization: Bearer $NARROW_TOKEN"
```

RESPONSE

```
HTTP/1.1 403

{
  "error": "insufficient_scope",
  "required_scope": "api.read",
  "error_description": "This endpoint requires the 'api.read' scope"
}
```

6 · Why each check matters

- **PKCE (S256)** — binds the authorization code to the client instance that started the flow. A stolen code cannot be redeemed without the matching `code_verifier`.
- **state** — returned unchanged on the `/callback` redirect so the client can reject cross-site request forgery.
- **nonce** — embedded in the ID token and checked by the client to prevent token replay.
- **Single-use code, 60s TTL** — the code is consumed atomically at the token endpoint, so replay fails even within the window.
- **aud separation** — the ID token is for the client, the access token is for the API; neither is accepted in the other's place.
- **Signature + iss/aud/exp on every API call** — the resource server trusts no token it cannot verify against the provider's JWKS, with no shared secret and no call back to the OP per request.

- **Refresh-token rotation** — refresh tokens are opaque and stored server-side, so the OP can *rotate* them (single-use) and *revoke* them instantly. Rotation makes a replayed refresh token a detectable failure (step 10a); server-side storage is what self-contained access-token JWTs deliberately give up.

Demo only. Secrets are hard-coded, users, keys and refresh tokens live in memory, transport is plain HTTP, and there is no consent handling. The point is to make every protocol step visible.